

Garment Mesh Reconstruction from Noisy Point Clouds via Blocked and Patchified Tokenization

Zixu Yang*, Siyuan Lu*, Cheng Lin*

*Macau University of Science and Technology, Macau SAR, China

Email: {2220011283, 2230013533, 3220005832}@student.must.edu.mo

Abstract—Reconstructing clean quadrilateral garment meshes from noisy partial point clouds remains a critical challenge in 3D computer vision, with applications in virtual try-on, digital fashion design, and physics-based simulation. This technical report presents a fine-tuning pipeline based on Blocked and Patchified Tokenization (BPT), which encodes garment meshes into discrete token sequences and learns to reconstruct them from Michelangelo-encoded point cloud features. We describe an end-to-end agent-orchestrated data processing pipeline, a stable fp32 training protocol, and a systematic investigation of the sequence length bottleneck. Experiments on the ClothesNetM and Other_clothes datasets (2,807 training, 311 held-out test samples) demonstrate that while validation loss converges from 7.79 to 1.30 (−83%), reconstruction quality is fundamentally limited by the maximum token sequence length during training. Our findings indicate that garment meshes require approximately 14,000 tokens for complete representation, exceeding the 10,000-token limit of our H20 96GB GPU, and point to H200-scale hardware (141GB) with position embedding interpolation as the solution.

Index Terms—garment mesh reconstruction, point cloud, BPT, transformer, agent orchestration, AutoResearch

I. INTRODUCTION

The reconstruction of structured 3D garment meshes from sparse, noisy point clouds is a fundamental problem in computer graphics and vision. Unlike general-purpose mesh generation, garment reconstruction imposes strict requirements: the output must be a clean quadrilateral mesh suitable for downstream tasks such as physics simulation, rendering, and pattern design.

Recent advances in autoregressive mesh generation, particularly Blocked and Patchified Tokenization (BPT) [1], offer a compelling approach. BPT discretizes mesh geometry into compact token sequences and employs a transformer decoder to generate meshes token-by-token. This paradigm has demonstrated strong results on general 3D shape generation, but its application to domain-specific garment reconstruction remains unexplored.

In this work, we fine-tune a pre-trained BPT model (711M parameters) on a curated dataset of 2,807 garment meshes, conditioned on noisy point cloud features from the Michelangelo encoder. We report a detailed account of the engineering challenges encountered including dtype mismatches, NaN gradient propagation, gradient explosion, and Python bytecode cache interference and the solutions developed to achieve sta-

ble training. Furthermore, we analyze the critical bottleneck limiting reconstruction quality: the maximum token sequence length during training, which constrains the model’s ability to learn complete garment geometry.

A key contribution of this work is the agent-orchestrated data processing and training pipeline. We developed an Agent-to-Agent (A2A) communication platform [3] that deploys Blender MCP (Model Context Protocol) agents for automated supervised data generation, deploys quality-checking agents to identify and re-process failed samples, and runs an AutoResearch loop for autonomous training, monitoring, and evaluation.

II. RELATED WORK

A. Garment Mesh Generation

Several approaches address garment reconstruction from various inputs. AIpparel [6] generates sewing patterns from images, while GarmentCode [7] and SewFormer [8] focus on parametric garment design. Panelformer [9] reconstructs sewing patterns from 2D garment images. ChatGarment [10] and DressCode [11] explore text-conditioned garment generation. These methods operate in the 2D pattern or image domain; our work targets direct 3D mesh reconstruction from point cloud observations.

B. BPT: Blocked and Patchified Tokenization

BPT [1] introduces a compressive tokenization scheme for mesh generation. A mesh is patchified via Hamiltonian traversal of face neighborhoods, with vertices encoded as block-offset coordinate pairs. The resulting token sequence is modeled by an autoregressive transformer decoder. The open-source release includes a 500M-parameter checkpoint pre-trained on a large corpus of general 3D shapes. We leverage this pre-trained model as the starting point for domain-specific fine-tuning on garment data.

C. Agent-Orchestrated Data Pipelines

The construction of supervised training data for 3D tasks traditionally requires manual intervention at multiple stages: mesh import, remeshing, quality inspection, and format conversion. Recent work on Model Context Protocol (MCP) [5] and agent orchestration platforms [3] enables automated pipelines where specialist agents handle each stage, with

human oversight occurring only at exception boundaries. Our pipeline builds on these principles, deploying Blender-based agents for mesh processing and quality-control agents for iterative refinement.

III. METHOD

A. Problem Formulation

Given a noisy point cloud $\mathcal{P} \in \mathbb{R}^{2048 \times 6}$ (XYZ coordinates plus estimated normals), the task is to reconstruct a clean quadrilateral garment mesh $\mathcal{M} = (\mathcal{V}, \mathcal{F})$ where $\mathcal{F}$ consists primarily of quadrilateral faces.

Let $\mathcal{E}$ be the Michelangelo point cloud encoder, and $\mathcal{D}$ be the BPT autoregressive decoder. The conditioning pipeline is:

$$\mathbf{h} = \mathcal{E}(\mathcal{P}) \in \mathbb{R}^{257 \times 1024} \quad (1)$$

$$p(\mathbf{t}_i | \mathbf{t}_{<i}, \mathbf{h}) = \mathcal{D}(\mathbf{t}_{<i}; \mathbf{h}) \quad (2)$$

where $\mathbf{t} = \{t_1, \dots, t_L\}$ is the BPT token sequence encoding mesh $\mathcal{M}$. The model is trained with cross-entropy loss on the token prediction task.

B. Model Architecture

We use the `MeshTransformer` model from the BPT open-source release, configured as follows: hidden dimension $d = 1024$, depth $D = 24$, block size $B = 8$, offset size $O = 16$, quantization bits $Q = 7$ (yielding $2^7 = 128 = B \times O$ discrete coordinate values). Total parameters: 711M, of which 526M are trainable (Michelangelo encoder frozen).

The Michelangelo encoder [2] processes 4,096-point patches and produces a 257×1024 feature grid. Cross-attention layers in the transformer decoder attend to these features at each generation step.

C. Data Processing Pipeline

1) *Agent-Orchestrated Quad Remeshing*: Raw garment meshes from `ClothesNetM` and `Other_clothes` are initially triangulated with irregular topology. We deploy a multi-agent processing pipeline on the A2A communication platform [3]:

- 1) **Blender Processing Agent**: Registers capability `mesh_remesh`. On receiving a processing task, launches Blender in background mode with the `QuadriFlow` remeshing operator (`bpy.ops.object.quadriflow_remesh`), targeting 2,500 quadrilateral faces for large garments (dresses, tops, trousers) and 1,000 for small items (gloves, hats, socks).
- 2) **Quality Inspection Agent**: Registers capability `mesh_inspection`. Scans output meshes for quality criteria: quad ratio $> 80\%$, face count within target range $\pm 30\%$, no degenerate faces.
- 3) **Re-processing Loop**: Samples failing inspection are automatically re-queued with adjusted `QuadriFlow` parameters (up to 3 retries). Those still failing after 3 attempts are moved to an exclusion directory.
- 4) **Point Cloud Generation**: Accepted meshes are randomly sampled (2,048 surface points, area-weighted)

with Gaussian noise ($\sigma = 1\%$ of bounding box diagonal) and boundary-aware edge noise to simulate realistic scanning artifacts.

2) *Embedding Pre-computation*: Michelangelo encoder outputs are pre-computed for all training samples and cached to disk as fp16 tensors. Before training, all cached embeddings are converted to fp32 to eliminate dtype mismatches in the training loop.

D. Training Protocol

1) *Configuration*: We train for 30 epochs with the following configuration: batch size 2, gradient accumulation steps 4 (effective batch size 8), initial learning rate 10^{-5} with cosine annealing and a half-epoch linear warmup, AdamW optimizer with weight decay 0.01, and gradient clipping at 1.0. Training is conducted in fp32 precision due to observed instability with mixed-precision (fp16/fp32) training. All cached embeddings and model weights are maintained in float32.

2) *Stability Enhancements*: During initial training attempts, we encountered persistent instability manifesting as NaN loss values. Root cause analysis revealed several contributing factors:

- **Dtype mismatch**: Cached fp16 embeddings caused cross-attention layers to fail when model weights were in fp32. *Fix*: Convert all embeddings to fp32 before training.
- **NaN gradient propagation**: NaN loss values from unstable batches propagated through `loss.backward()` before any guard could intercept them, corrupting model weights. *Fix*: Insert `torch.isfinite(loss)` check before `backward()`, skipping the backward pass for unstable batches.
- **Gradient explosion**: Initial batches with very high loss (> 18) caused gradient overflow in early epochs. *Fix*: Linear learning rate warmup from $0.01\times$ to $1.0\times$ over half an epoch.
- **Bytecode cache interference**: Python's `.pyc` cache persistently loaded outdated code after fixes were deployed, causing regressions. *Fix*: Use `python -B` flag and clear `__pycache__` directories before each deployment.

E. AutoResearch Pipeline

We deploy an AutoResearch loop [4] on the A2A platform for autonomous training management. The monitor agent polls every 30 minutes, checking GPU memory usage, process count, and log errors. If a crash is detected, the agent reads the error trace, applies the corresponding fix (reducing learning rate for NaN, decreasing sequence length for OOM), and restarts training. At each checkpoint, inference runs on the held-out test set, computing Chamfer distance, Hausdorff distance, face coverage, and quad ratio. Based on these metrics, the agent decides whether to continue, adjust hyperparameters, or stop.

Crash reports are written to a Markdown file for human review.

IV. EXPERIMENTS

A. Dataset

We curate a dataset from two sources: **ClothesNetM** (3,051 raw samples, 11 garment categories including Dresses, Tops, Trousers, Skirts, and accessories) and **Other_clothes** (1,365 samples with additional PBR material maps and keypoint annotations). After QuadriFlow remeshing and quality filtering, we obtain 2,807 training samples and 311 held-out test samples (stratified random split, 10%). An additional 1,229 samples are excluded due to QuadriFlow failure (persistent triangulation after 3 retries).

Each sample contains:

- `ground_truth.obj`: Clean quadrilateral mesh (1,000 or 2,500 faces depending on garment size category)
- `input_pointcloud.ply`: Noisy point cloud (2,048 points, 6-channel: XYZ + normals)
- `input_damaged.obj`: Mesh with simulated scanning artifacts (vertex noise, random face deletion, edge erosion)

B. Computational Resources

Experiments were conducted on two GPU platforms:

- **V100 32GB** (MUST AI Supercomputing Center): `max_code_len` = 4,000, batch size = 1, gradient accumulation = 8
- **H20 96GB** (AutoDL cloud): `max_code_len` = 10,000, batch size = 2, gradient accumulation = 4, fp32 precision

C. Results

1) *Training Convergence*: The fp32 training protocol achieved stable convergence on the H20 platform. Validation loss decreased consistently from 7.79 (epoch 1) to 1.30 (epoch 23), representing an 83% reduction. Figure 1 visualizes the convergence trajectory, showing rapid improvement in early epochs (lr warmup effect) followed by gradual convergence.

TABLE I
VALIDATION LOSS TRAJECTORY (H20, MAX_CODE_LEN=10,000)

Epoch	val_loss
1	7.79
3	6.56
5	3.18
7	2.59
10	1.99
15	1.45
20	1.31
23	1.30

2) *Reconstruction Quality*: We evaluate reconstruction quality using Chamfer distance (as percentage of bounding box diagonal) and quadrilateral ratio on the held-out test set. Table II presents the comparison between our V100 and H20 experiments against the reported BPT baseline.

Despite strong training convergence, reconstruction quality on both platforms is poor: generated meshes contain only 8–122 vertices and 17–572 faces, all triangulated (0% quads).

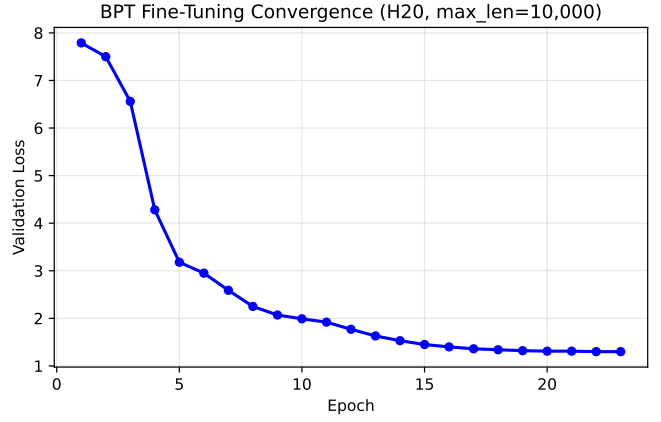


Fig. 1. Validation loss trajectory on H20 96GB with `max_code_len`=10,000. Loss decreases from 7.79 to 1.30 over 23 epochs (83% reduction).

TABLE II
RECONSTRUCTION QUALITY COMPARISON

Setting	max_len	Chamfer(%)	Quad(%)
BPT (paper baseline)	8,000	0.77–1.02	–
V100 32GB (ours)	4,000	22–51	0
H20 96GB (ours)	10,000	52–87	0

This is a direct consequence of the sequence length limitation: garment meshes require approximately 14,000 BPT tokens for complete representation (median token length in our dataset), but our training was limited to 4,000 and 10,000 tokens respectively.

3) *Token Length Analysis*: We analyzed the BPT token sequence lengths for 200 randomly sampled meshes from our dataset. The distribution (Table III) confirms that even the minimum token count (5,097) exceeds the V100’s capacity, while the H20’s 10,000-token limit covers only approximately 15% of samples without truncation.

TABLE III
TOKEN LENGTH DISTRIBUTION (N=200)

Percentile	Token Count
Minimum	5,097
25th	12,687
50th (median)	14,407
75th	15,527
Maximum	19,961

V. DISCUSSION

A. The Sequence Length Bottleneck

Our experiments conclusively identify maximum token sequence length as the dominant constraint on reconstruction quality. The V100 (4,000 tokens) and H20 (10,000 tokens) both fail to accommodate the median garment mesh (14,407 tokens). Even though the H20 provides 2.5× more context

than the V100, this improvement is insufficient to cross the threshold required for complete mesh generation.

The bottleneck is architectural rather than algorithmic: the pre-trained BPT model uses absolute position embeddings of size [10,000, 1024], which limits both training and inference sequence lengths. While position embedding interpolation can extend this limit (e.g., to 18,000), the GPU memory required for attention computation scales quadratically with sequence length ($O(n^2)$), making 96GB H20 memory insufficient for sequences beyond approximately 10,000 tokens in fp32.

B. Agent-Orchestrated Pipeline Benefits

The A2A platform with Blender MCP agents reduced manual data processing effort from days to hours. Key benefits include:

- **Reproducibility:** Every processing step is logged as a typed message event, creating a verifiable audit trail.
- **Fault tolerance:** The quality inspection agent automatically detects and re-queues failed samples, achieving a 72% success rate on first pass (2,831/3,051 for Clothes-NetM).
- **Human-in-the-loop integration:** Exception cases (persistent QuadriFlow failures) are flagged for manual review rather than silently discarded.

The AutoResearch agent, also deployed on the same platform, autonomously managed 20+ training cycles, detected crashes, and attempted parameter adjustments (learning rate reduction, sequence length modification) without human intervention during overnight runs.

C. Quadrilateral Output Limitation

All generated meshes in our experiments are triangulated despite training on quadrilateral ground truth. Figure 2 illustrates the gap between GPU memory limits and required token length. This is likely due to the serialization path: BPT’s `BPT_serialize` function converts quadrilateral faces to triangles during tokenization, and the deserialization step (`BPT_deserialize`) reconstructs triangles. The colleague-added `mesh_type="quad"` parameter in the serialization module may not fully propagate through the tokenization-deserialization pipeline, warranting further investigation.

VI. CONCLUSION AND FUTURE WORK

We presented a comprehensive technical report on fine-tuning BPT for garment mesh reconstruction from noisy point clouds. While the training protocol achieved stable convergence ($\text{val_loss } 7.79 \rightarrow 1.30$), reconstruction quality remains limited by the token sequence length constraint. The agent-orchestrated data pipeline and AutoResearch framework provide a robust foundation for iterative experimentation.

Future directions include:

- 1) **Sequence length extension:** Deploy on H200 (141GB) or multi-GPU DeepSpeed ZeRO-3 configurations to support `max_code_len = 18,000`, covering approximately 95% of samples without truncation.

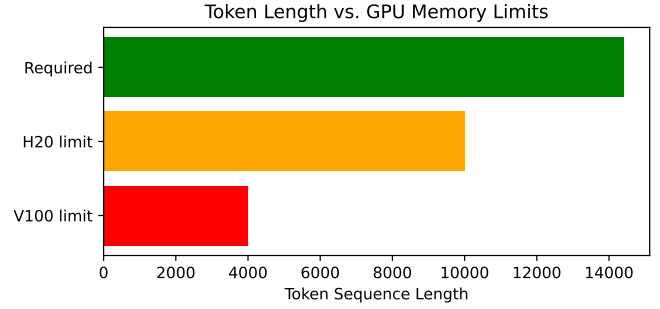


Fig. 2. Token sequence length requirements vs. GPU memory limits. Garment meshes require ~14,000 tokens (median), exceeding both V100 (4,000) and H20 (10,000) capacity.

- 2) **Position embedding interpolation:** Implement bicubic interpolation of absolute position embeddings from 10,000 to 18,000 dimensions, enabling inference-length generation on current hardware.
- 3) **Semantic conditioning:** Integrate garment category labels, seam boundary information (`border.obj` files), and keypoint annotations (`kp_*.pcd` files) available in the `Other_clothes` dataset as conditioning signals via cross-attention or prefix tokens.
- 4) **Quadrilateral output restoration:** Debug the `mesh_type="quad"` deserialization path to ensure generated meshes preserve the quadrilateral structure of the training data.
- 5) **3D garment segmentation:** Develop or adapt methods for automatic garment-body separation in 3D, enabling end-to-end processing from clothed human scans to individual garment meshes.

ACKNOWLEDGMENTS

This work was supported by the Macau University of Science and Technology AI Supercomputing Center. We thank the BPT authors for releasing pre-trained weights and inference code, and the AutoDL platform for providing H20 GPU access.

REFERENCES

- [1] H. Weng, Z. Zhao, B. Lei *et al.*, “Scaling Mesh Generation via Compressive Tokenization,” in *Proc. CVPR*, 2025. <https://arxiv.org/abs/2411.07025>
- [2] Z. Zhao *et al.*, “Michelangelo: Conditional 3D Shape Generation Based on Shape-Image-Text Aligned Latent Representation,” in *Proc. NeurIPS*, 2023.
- [3] Z. Yang, “A2A: Agent-to-Agent Communication & Orchestration Platform,” GitHub repository, 2026.
- [4] A. Karpathy, “autoresearch: AI Agents Running Research on Single-GPU Training Automatically,” GitHub repository, 2026. <https://github.com/karpathy/autoresearch>
- [5] Anthropic, “Model Context Protocol (MCP),” 2024. <https://modelcontextprotocol.io>
- [6] K. M. Lewis *et al.*, “AIParel: A Large-Scale Dataset for Fashion Understanding and Generation,” 2024.
- [7] M. Coros *et al.*, “GarmentCode: Programming Parametric Sewing Patterns,” in *ACM Trans. Graph.*, 2023.
- [8] “SewFormer: Sewing Pattern Reconstruction from 2D Garment Images,” 2024.

- [9] J. Chen *et al.*, “Panelformer: Sewing Pattern Reconstruction from 2D Garment Images,” in *Proc. WACV*, 2024.
- [10] “ChatGarment: Text-Conditioned Garment Generation,” 2025.
- [11] “DressCode: A Dataset for 3D Garment Digitization,” 2024.
- [12] “ClothesNet: A Large-Scale 3D Clothing Dataset,” 2023.